

Drone Security Analyst Agent

Project Report — AI Engineer Assignment

1. Introduction

This report documents the design, development, and evaluation of a Drone Security Analyst Agent prototype. The system processes simulated drone telemetry and video frame data to detect security events, generate alerts, and produce intelligent surveillance summaries for property monitoring.

2. Problem Statement

A docked drone monitors a fixed property daily. The agent must:

1. Process simulated telemetry data and video frames in real-time
2. Identify objects and activities and log them with context
3. Generate immediate alerts based on predefined security rules
4. Index video frames in a searchable format for future reference

3. Assumptions

1. Video frames are represented as text descriptions simulating drone camera output. Real camera integration is outside the scope of this prototype.
2. Telemetry data is matched to frames by timestamp.
3. Late night hours are defined as 22:00 to 06:00 for alert triggering purposes.
4. A vehicle appearing two or more times in the same day is flagged as a repeated vehicle event.
5. A person detected at the same location across two or more frames is considered loitering.
6. Gate tampering is detected through keyword matching in frame descriptions.
7. The Groq free tier rate limits are sufficient for the prototype demonstration.
8. No GPU is required as all LLM inference is handled via the Groq API.

4. System Architecture

The system is organized into four layers:

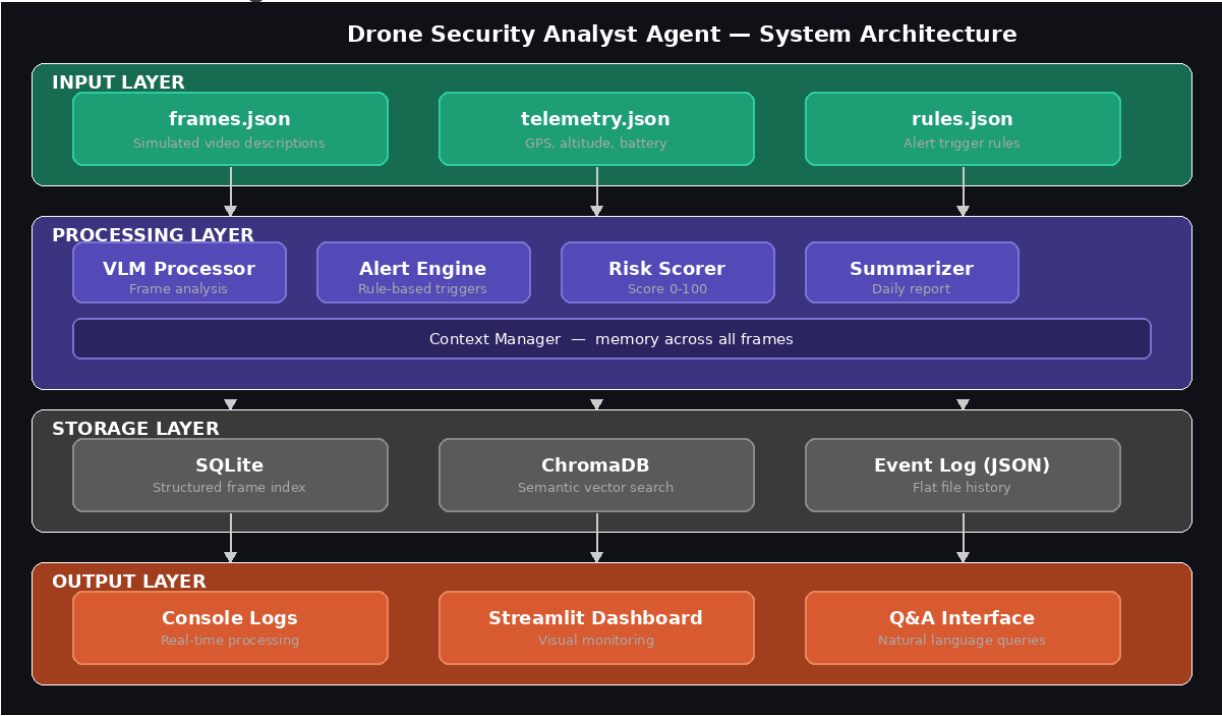
Input Layer — Simulated data files representing drone video frames, GPS telemetry, and predefined alert rules.

Processing Layer — Core AI components including the VLM processor for frame analysis, alert engine for rule-based detection, risk scorer for per-frame threat quantification, pattern detector for cross-frame analysis, and a context manager that maintains memory across all frames.

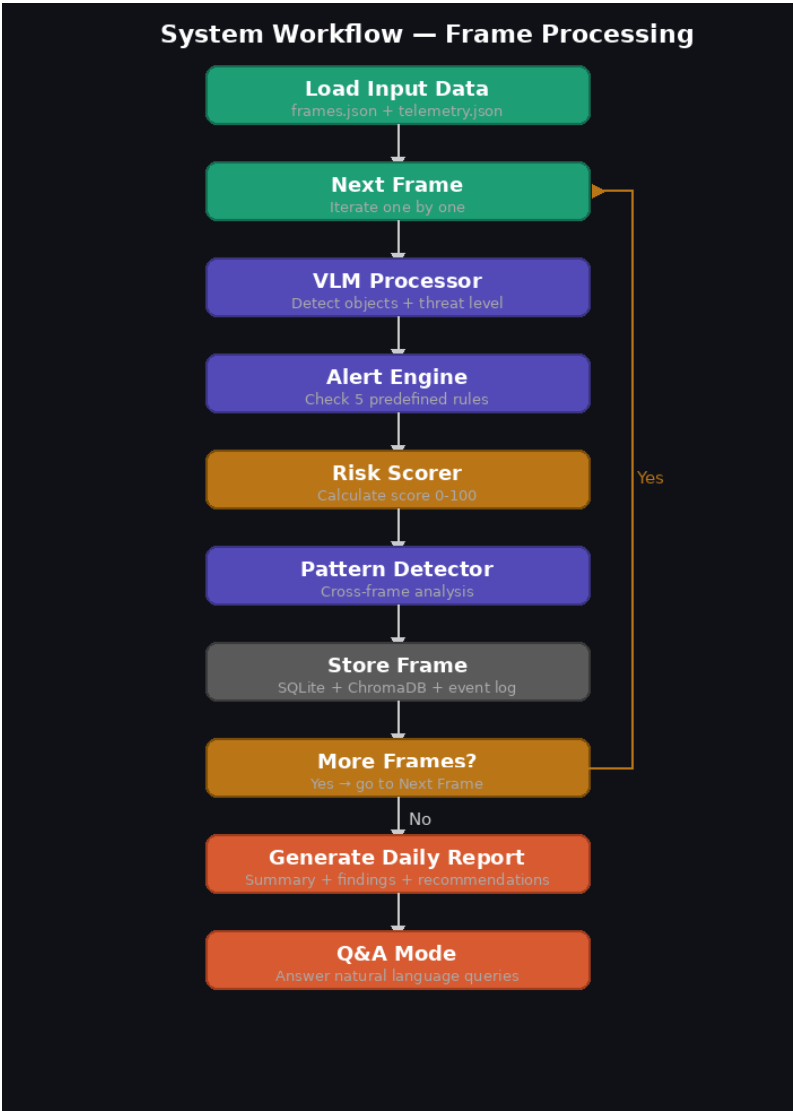
Storage Layer — SQLite for structured frame indexing, ChromaDB with sentence-transformers for semantic vector search, and a JSON event log for flat file history.

Output Layer — Console logs for real-time processing output, Streamlit dashboard for visual monitoring, daily report generation, and an interactive Q&A interface.

Architecture Diagram:



System Workflow:



5. Tool and Technology Choices

Groq API — Qwen3-32B

Groq was selected for its free tier with fast inference speeds. Qwen3-32B was chosen as it is a production-grade model on Groq, distinct from Llama-based models used in prior projects. It provides strong reasoning capabilities suitable for security analysis tasks.

LangChain

Used for agent orchestration and context management. LangChain is the industry standard framework for building LLM-powered agents and provides clean abstractions for memory and tool integration.

SQLit

Built into the Python standard library — requires no external server setup. Provides structured querying by time, location, and alert level.

ChromaDB with sentence-transformers

ChromaDB was chosen for its simpler API and persistent storage. The all-MiniLM-L6-v2 embedding model runs fully locally without any API cost, enabling semantic search queries where exact keyword matching would be insufficient.

Streamlit

Enables rapid Python-based UI development without requiring a separate frontend framework. Appropriate for a prototype where demonstrating functionality is the priority.

Rule-based alerts alongside LLM

LLM threat assessment can vary across runs. Rule-based alerts provide deterministic, auditable triggers that always fire under defined conditions regardless of LLM output. Both systems run in parallel.

6. Innovation Features

Beyond the core requirements, the following features were added:

Risk Scoring (0-100 per frame)

Each frame receives a numerical risk score based on time of day, location, detected objects, behavior keywords, and active alerts. This provides a quantifiable threat level beyond categorical labels.

```
[RISK] Frame 6 | Score: 85/100 | Reason: Night time + Gate location + Tampering detected  
[RISK] Frame 5 | Score: 0/100 | Reason: Daytime, normal activity
```

Pattern Detection

The system detects cross-frame patterns such as repeated vehicle visits, repeated person detections, and high-frequency alert locations.

```
[PATTERN] Main Gate has triggered activity 3 times today — high risk zone [PATTERN]  
Vehicle has appeared 2 times today
```

Daily Summary Report

After all frames are processed, the agent generates a comprehensive report including a statistical summary, a natural language surveillance summary, key findings, and actionable security recommendations. The report is saved to storage/daily_report.txt and displayed on the Streamlit dashboard.

Surveillance Summary (Bonus)

A 2-3 sentence natural language summary of all surveillance activity is generated as part of the daily report. This is a text-based summary generated by the LLM from all processed frame data — not a video.

Follow-up Q&A; Interface (Bonus)

Both the terminal and Streamlit dashboard support natural language questions about the day's surveillance data. Queries are answered using semantic search over ChromaDB combined with LLM reasoning.

```
Question : Who was attempting to open the main gate? Answer : An unknown person in dark clothing was attempting to open the main gate at 23:58, a critical security event.
```

7. Results

Sample console output from one frame:

```
[LLM] Objects : person [LLM] Summary : A person in dark clothing is standing near the main gate and looking around suspiciously. [LLM] Threat : MEDIUM [ALERT] HIGH — Activity detected at Main Gate, late night hours [RISK] Frame 1 | Score: 75/100 | Night time + Gate + Suspicious behavior
```

Daily summary (generated automatically):

```
Total frames analyzed : 6 Total alerts : 8 Critical alerts : 1 High alerts : 6 Most active location : Main Gate
```

Surveillance summary (generated automatically):

Today's surveillance revealed suspicious activity at the Main Gate, including a person in dark clothing loitering repeatedly during early morning and late-night hours, culminating in a possible gate breach attempt at 23:58. A blue Ford F150 truck entered the Garage twice, with its second visit occurring midday. Only routine activity was observed in the backyard during afternoon hours.

8. What Could Be Done Better

Real Video Processing — With more time, actual video frames could be processed using a vision-language model such as BLIP-2 or LLaVA. This would replace text descriptions with genuine visual analysis.

Persistent Learning — The current prototype uses in-context memory which resets between sessions. A persistent feedback mechanism with database-backed alert history would allow the system to improve over time.

Telegram Alert Integration — The notification module is implemented and ready. With additional time, real-time Telegram alerts would be fully integrated for instant property owner notifications.

Scalability — The current system processes frames sequentially. A production system would use async processing, a message queue, and a more scalable vector database such as Pinecone or Weaviate.

More Alert Rules — The current rule set covers five scenarios. A production deployment would include crowd detection, unusual movement patterns, and time-based access control violations.

9. AI Tools Used

Claude (Anthropic)

Used throughout the development process for architecture design, code generation, debugging, and documentation. Claude generated initial boilerplate for the LangChain agent, alert engine, Streamlit dashboard, and test cases. All generated code was reviewed, customized, and integrated manually.

Groq API — Qwen3-32B

Powers the core frame analysis, daily summary generation, and Q&A; responses within the running system.

Impact on Workflow

AI-assisted development reduced time spent on boilerplate and repetitive code, enabling focus on system design, prompt engineering, and innovation. Architecture decisions were validated through AI-assisted reasoning, and edge cases in alert logic were identified through collaborative debugging.

10. References

1. Groq API Documentation — <https://console.groq.com/docs>
2. LangChain Documentation — <https://python.langchain.com>
3. ChromaDB Documentation — <https://docs.trychroma.com>
4. Streamlit Documentation — <https://docs.streamlit.io>
5. GitHub Repository - <https://github.com/Shuzi20/drone-security-agent>

